Alexandria Production Blueprint and Runbook

Shared-memory provider, SQLite integrity, backup and recovery

25 October 2026 | Revision 3.4 - Lean Production Readiness

Contents

- Start here: reviewed baseline and decisions
- Architecture and data contract
- Priority tiers and canonical build order
- A0 - Reproduce baseline and inventory data
- A1 - Remove JSON restore and keep one full-restore path
- A2 - Verify SQLite backup and offline restore
- A3 - Enforce active deduplication in SQLite
- A4 - Maintain migrations and schema compatibility
- A5 - Make package/source sync repeatable
- A6 - Recovery drill and Raphael integration
- Commands and source map
- Official technical references

This runbook is the lean implementation source of truth for taking Alexandria, the Python shared-memory library, from the reviewed passing-CI baseline to a verified personal deployment. Alexandria remains an in-process Python module with its own SQLite database file; it is not a separate HTTP service. Raphael owns Android, network access, model routing, and the phone-to-laptop integration path.

> This document is a plan, not a certification. Passing CI does not prove that a real database was restored, that the laptop's backup files are recoverable, or that the production phone/gateway integration works.

0. Start here

0.1 Reviewed repository baseline

- Repository: Agri99/Alexandria
- Default branch: main
- Reviewed HEAD: 0fffa6667aeab05bf0d8a8cd2587afe4ce5abc23 (refactor(sqlite_provider): extract row-to-record helper and clean redundant rules)
- Prior hardening commit: 2f0223157b0610158e9568b3889a047d977e9175
- Latest observed Alexandria CI at review: run 40089329716 - success
- Raphael gateway reviewed HEAD: 43dd995462963fb3191db95334a0e57dbc9bd982; its Gateway CI includes an authoritative Alexandria checkout and parity check.
- Current workflow and package metadata still include Python 3.10. Python 3.10 reached end-of-life in October 2026; revise the supported-runtime declaration/matrix once dependencies and code are verified on a maintained version set.

Current strengths include an explicit MemoryProvider boundary, SQLite Online Backup API support, schema version metadata, transaction-oriented migrations, body normalization/hash validation, request-id

idempotency, soft-delete/undo behavior, and tests for provider validation, migrations, backup, and concurrency. The principal simplification for this revision is to stop treating JSON snapshot restore as a production capability that must be hardened.

0.2 Current risk register

Tier 1 - Must

- **Two restore mechanisms:** `restore_json_snapshot()` duplicates restore semantics and creates a second content/conflict policy. Remove it unless a real caller requires it; SQLite Online Backup is the sole full-database restore path.
- **Active duplicate race:** the provider's Python lock is scoped to one provider instance. Clean pre-existing active duplicates, then enforce the invariant in SQLite with a partial unique index for `status='approved'` rows.
- **No recorded restore drill:** create and verify a backup, restore it offline to a disposable target, validate schema/integrity, and perform a read/write smoke test.
- **Embedded source drift:** make the sync script remove stale embedded modules safely and retain the byte-for-byte parity test against the authoritative standalone package.

Tier 2 - Should

- **EOL runtime in CI:** Python 3.10 remains in workflow and package metadata even though it reached EOL in October 2026. Validate maintained versions, then update both Alexandria and Raphael Gateway CI in one work batch.
- **Future migration safety:** keep forward-only transaction-safe migrations and strengthen populated-database upgrade/failure fixtures before introducing a new schema version.

Tier 3 - Could defer

- Automated retention schedulers, formal RPO/RTO documents, JSON import/restore, vector retrieval, and multi-user architecture are not required for the current personal deployment. A practical backup location and one successful recovery drill are required; a large operations program is not.

1. Architecture and data contract

1.1 Keep this architecture

- `Agri99/Alexandria` is the canonical source for the `alexandria` Python package.
- Raphael currently embeds a synchronized copy under `Raphael/src/alexandria`; the gateway CI checks the embedded copy against a checkout of this repository.
- SQLite is the backing store. The live database is a separate file (default path configured by Raphael is `data/alexandria.sqlite3`) and must never be committed.
- `MemoryProvider` is the application boundary; clients should not access SQLite directly.
- The Raphael restricted gateway is a separate process from F.R.I.D.A.Y.'s desktop/voice runtime. Alexandria itself does not need a separate process, port, TLS stack, or public API for the current single-user release.
- Use SQLite WAL plus the SQLite Online Backup API. Do not introduce PostgreSQL, an ORM rewrite, vector storage, cloud sync, or a new Alexandria service without an explicit need and measured evidence.

1.2 Preserve these memory semantics

1. Only `status='approved'` rows may be returned by `load_context()`.

2. A deliberate Save/Remember action by the user is approval. Do not introduce model-generated automatic memory extraction in this pass.
3. A memory is a concise fact, preference, or project detail - not a conversation transcript, prompt dump, voice recording, or credential.
4. Reusing the same `client_request_id` with the same payload is idempotent. Reusing it with conflicting content returns a typed conflict.
5. Active exact duplicates use normalized content and `body_hash`. A deleted row is not active; restore must not create an active duplicate.
6. Edits apply normalization, secret/transcript policy, maximum length, and duplicate checks just like creates.
7. Soft-deleted records are excluded from list/context immediately. Restore is bounded by the configured undo window and duplicate rules. Expired deleted rows remain stored unless a separately reviewed purge command is implemented; do not silently add a background purge.
8. JSON export, if retained, is a human-readable export. It is not a full database backup and must not be marketed as a restorable snapshot.
9. SQLite backup preserves a consistent database state including schema and deleted rows. Restore is an operator-level offline operation.
10. Memory retrieved into a prompt is untrusted reference data, never a system instruction or permission grant.

1.3 Three priority tiers

- **Tier 1 - Must:** remove avoidable restore complexity, protect the active-dedup invariant in SQLite, prove backup/restore, keep repository parity, and pass the lifecycle/integration tests. These block a production claim.
- **Tier 2 - Should:** update supported Python versions, strengthen migration tests, and document practical backup habits/source-of-truth rules. These improve supportability without creating a new storage subsystem.
- **Tier 3 - Could:** automated retention schedulers, elaborate RPO/RTO service documents, JSON import, a vector database, automatic extraction, or multi-user architecture. These are explicitly out of scope absent new requirements.

2. Priority tiers and canonical build order

Complete tasks in this sequence and keep the scope narrow:

1. **A0 - Baseline:** confirm checkout, supported runtime, test status, data path, and current schema on a disposable copy.
2. **A1 - Remove JSON restore:** remove `restore_json_snapshot()` and associated public/API/CLI call paths and tests. Keep JSON export only if useful; otherwise remove both formats rather than preserving unused surface area.
3. **A2 - SQLite restore:** validate the Online Backup API output; require all processes to stop before restore; preserve the pre-restore target; validate schema/integrity before and after; perform one recovery drill.
4. **A3 - Active dedup:** find/resolve any active duplicates using a reviewed rule, then add a partial unique index on `body_hash` for active (`approved`) rows and tests for independent provider instances/connections.
5. **A4 - Migrations:** keep forward-only ordered migrations, fail closed on unknown future schema versions, and test on populated copies.
6. **A5 - Source sync/package:** keep the standalone repository authoritative; delete stale embedded Python modules during sync safely; enforce parity and install/test the built package in a clean environment.
7. **A6 - Release integration:** run lifecycle, backup/restore, privacy, and Raphael gateway integration checks; write down what has actually been tested.

The gates below contain Must and Should tasks. A long operational-documentation project is not a substitute for passing the core data invariants.

3. A0 - Reproduce baseline and inventory data

Step 1 - Preserve the working tree and record source identity

```
Set-Location C:\Dev\Alexandria
git status --short
git log -5 --oneline
```

Inspect local changes. Do not reset, overwrite, or run sync into a dirty Raphael tree without reviewing the diff. Record the SHA and schema version in a local checklist. Do not record actual memory bodies.

Step 2 - Test on a maintained Python version

Use Python 3.12 as the local default, or another supported version required by the deployment:

```
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e .
python -m pip install -r requirements.txt
pytest -q
```

After validating dependency compatibility, update `pyproject.toml` (`requires-python`) and `.github/workflows/alexandria.yml` to the chosen maintained versions. Proposed starting CI matrix: Python 3.11, 3.12, and 3.13; adjust if a dependency or explicitly supported runtime requires a different set. Python 3.10 should not remain a maintained target after EOL.

Step 3 - Inspect schema safely

Use a disposable database copy or a read-only inspection command; do not inspect/copy an active WAL database by manually copying just the main `.sqlite3` file. Confirm:

- `PRAGMA integrity_check` returns `ok`;
- `schema_meta` exists and the schema version is understood;
- required tables, columns, and indexes exist;
- counts by `status` and `is_core` are plausible;
- active duplicate `body_hash` values are enumerated before adding a unique index;
- `.sqlite3`, `-wal`, `-shm`, export files, temporary files, and backups are ignored by Git and stored with user-only permissions.

Never paste real memory contents into an issue or test report. Use synthetic IDs and aggregate counts.

A0 gate

Must: local tests pass on the chosen deployment runtime; current SHAs and schema version are known; no live DB is exposed to test mutations. **Should:** confirm the maintained CI matrix works before changing package metadata.

4. A1 - Remove JSON restore and keep one full-restore path

Why removal is the preferred fix

The existing src/alexandria/backup.py includes SQLite backup/restore and JSON snapshot functions. JSON restore duplicates safety decisions already handled by normal writes and creates ambiguity between "import records" and "replace the database." A correctly hashed snapshot is not authorization, and INSERT OR REPLACE can overwrite conflicting records. For this personal, single-user deployment, the less risky design is to remove the restore path rather than build a second import product.

Step 1 - Inventory all call paths before deleting

Search the standalone Alexandria repo and Raphael repo for:

rg "restore_json_snapshot|validate_json_snapshot_data|MemoryExport|json snapshot|import.*snapshot" .

If rg is not installed, use your editor's repository-wide search. Include source, tests, CLI commands, README/docs, gateway routes, and sync scripts. Do not delete JSON export models or serialization code until you have decided whether a human-readable export is still useful.

Step 2 - Make restore semantics explicit by removing import

- Delete restore_json_snapshot() and any public wrappers, CLI options, HTTP routes, or API claims that expose JSON restore.
- Remove tests whose only purpose was to preserve JSON restore behavior; replace them with tests that prove JSON export is export-only and does not mutate a database, if export remains.
- If validate_json_snapshot_data() only supports restore, remove it too. Keep reusable content validators that are still used by normal writes.
- Retain create_sqlite_backup() and restore_sqlite_backup() as the full database backup/restore mechanisms.
- Update README/API documentation: JSON output is a readable export, not a restorable backup; full restore requires an offline SQLite backup/restore procedure.
- Verify no stale route/CLI path remains and that the package API no longer exposes JSON restore accidentally.

Do not keep a "temporary" compatibility shim unless a real external caller exists. If one does exist, make it fail explicitly with a deprecation error and remove it in a documented later version rather than silently mutating data.

Step 3 - Test removal and export behavior

- Search tests/source/docs again after deletion; the only mentions of JSON restore should be the explicit explanation that it is unsupported.
- If JSON export remains, verify the export schema is human-readable and includes only the intended records (currently approved facts), and confirm export has no write side effect on the database.
- Verify an attempted invocation of a removed CLI/API entry point fails clearly rather than falling through to a different operation.
- Run all Alexandria and Raphael Gateway tests after the standalone source changes and before synchronization.

A1 gate

Must: no live JSON restore implementation or reachable entry point remains; SQLite is the only full-database restore path; docs do not imply that JSON exports can restore a database. **Should:** remove dead validation/models/docs made unused by this decision.

5. A2 - Verify SQLite backup and offline restore

Primary files:

- src/alexandria/backup.py
- src/alexandria/migrations.py
- src/alexandria/sqlite_provider.py
- tests/test_backup.py
- the Raphael gateway CLI/operations documentation that calls restore

Step 1 - Backup while writes may exist; restore only while offline

create_sqlite_backup() uses the SQLite Online Backup API and is the correct mechanism for a consistent live snapshot, including when WAL writes are active. Continue using it rather than copying the live .sqlite3 file by hand.

Restore has a different precondition: stop the gateway and every other process using the database. A lock probe is helpful but is not proof that no idle connection/process remains. Never restore over a live database.

Step 2 - Treat backup files as untrusted until verified

For each backup candidate:

1. Confirm the file exists and has non-zero size.
2. Open it read-only for validation when supported by the current implementation.
3. Run PRAGMA integrity_check; require ok.
4. Verify schema_meta and reject schema versions newer than the code supports.
5. Verify required tables, columns, and indexes exist.
6. Optionally compute SHA-256 and record it beside the file.
7. Record timestamp, schema version, aggregate record counts, validation result, and checksum - never body text or tokens.

The restore routine should validate the source before touching the target. For an existing target, create a verified pre-restore backup or rename/copy it to a rollback path before replacement. If validation fails at any step, leave the target unchanged.

Step 3 - Validate target replacement safely

- Resolve source/target paths to absolute paths before work begins.
- Reject identical source and target paths.
- Require target offline; if a lock is detected, fail closed with an actionable message.
- Keep temporary restore files in the same filesystem where atomic replacement is expected.
- Check/handle -wal and -shm sidecars only after every connection is closed; do not remove active sidecars as a substitute for shutting the gateway down.

- After replacing the target, reopen it through the normal provider, run integrity/schema checks again, and perform a read/write smoke test on a disposable restore target.
- If post-restore checks fail, do not label restore successful; preserve the failed artifact for diagnosis and restore the pre-restore target when safe.

Step 4 - Failure tests

Cover at least:

- missing source file;
- empty/corrupt SQLite file;
- source without `schema_meta` or with invalid version;
- unsupported future version;
- missing required table/column;
- locked/live target;
- source and target path identical;
- successful restore to a new destination;
- restore over an existing target with rollback copy;
- failed verification leaves the target unchanged;
- read/write smoke test after restore.

A2 gate

Must: one recovery drill succeeds on a disposable target; source integrity/schema are checked; the gateway and all DB clients are stopped before target replacement; invalid restore leaves the target unchanged. **Should:** make operator output include checksum, schema version, and aggregate counts.

6. A3 - Enforce active deduplication in SQLite

Step 1 - Inspect and resolve current active duplicates

The current provider has an instance-local lock around duplicate handling. That protects competing calls through the same provider object but does not guarantee uniqueness across two provider objects or processes. First inventory the database before adding the constraint.

On a disposable copy, group approved rows by `body_hash` and report duplicate groups with synthetic IDs/counts. For each group, decide which existing record to keep based on the documented canonicalization/lifecycle rules. Do not auto-delete live data based only on a hash report; compare normalized content on a protected local copy, preserve core flags where appropriate, and take a verified backup before any cleanup.

Step 2 - Add a partial unique index in a new migration

After duplicate cleanup and tests, add a new forward-only migration in `src/alexandria/migrations.py`. The intended invariant is unique active/approved rows per normalized `body_hash`, while allowing a deleted row and a new active row with the same body. SQLite supports partial unique indexes, conceptually:

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_memories_active_body_hash
ON memories(body_hash)
WHERE status = 'approved';
```

This is the invariant; the implementation must use the project's migration transaction style and naming conventions. A partial unique index is preferable to an in-memory lock as the final guard. Keep the Python lock if it makes error handling cleaner, but don't rely on it for correctness.

Translate SQLite uniqueness failures into the existing typed duplicate/conflict result expected by callers. Do not leak raw SQL exceptions through the HTTP API. Ensure restore/undo of a deleted record hits the same database constraint and returns a stable duplicate conflict instead of resurrecting two active records.

Step 3 - Test with independent connections/providers

- Two provider instances attempt to create the same normalized body at the same time; at most one active row commits.
- Two provider instances use different request IDs for the same active body; no duplicate active row.
- Same request ID/same payload is idempotent.
- Same request ID/different payload is a conflict.
- A deleted row and a new approved row with the same body can coexist under the intended policy; restoring the deleted row returns a duplicate conflict.
- Normalization/hash remain stable for Unicode normalization and whitespace variants.
- Migration from existing schema v1 with active duplicate rows has a documented safe failure/cleanup path; the migration must never silently delete memory rows in order to succeed.

A3 gate

Must: a database-level uniqueness invariant exists for active memories; duplicate cleanup is explicit/backed up; tests prove independent-instance races cannot commit duplicate approved rows. **Should:** add a clear one-worker configuration assertion to the gateway so deployment cannot accidentally scale to multiple workers without a new concurrency review.

7. A4 - Maintain migrations and schema compatibility

Primary files: src/alexandria/migrations.py, src/alexandria/sqlite_provider.py, tests/test_migrations.py, tests/test_backup.py.

- Keep migration versions strictly increasing and migrations forward-only in normal deployment.
- Each migration should be atomic and update schema metadata only when all its statements succeed.
- A migration error must roll back its own work and leave the previous schema/version usable.
- Fail closed if the database declares a schema version newer than the running package supports.
- Do not use CREATE TABLE IF NOT EXISTS alone as proof that an existing table has the expected shape; verify required columns/indexes explicitly.
- Keep old-schema fixtures or create disposable v1 databases in tests. Before adding v2, test empty initialization, populated upgrade, duplicate-data handling, forced migration failure, reopen, and backup restore on the upgraded schema.
- Take a verified SQLite backup before an operator-triggered schema migration against real data. Do not automatically run a destructive downgrade.

A4 gate

Must: current migration path is transaction-safe and rejects unsupported schema versions. **Should:** maintain representative old-schema fixtures and test backup/restore across each released schema version as versions accumulate.

8. A5 - Make package/source sync repeatable

Step 1 - Keep one authoritative source

Agri99/Alexandria is canonical. Raphael's src/alexandria copy is a deployment embedding, not a second place to hand-edit memory behavior. Make changes in Alexandria first, then synchronize into Raphael.

Step 2 - Make sync remove drift, not just copy new files

Current sync behavior should be reviewed for stale Python modules. A deterministic sync operation should:

1. identify the canonical package directory;
2. copy current authoritative .py/required package files;
3. remove obsolete embedded source files that no longer exist in canonical source, within the intended destination only;
4. preserve Raphael-specific surrounding code/config/data and never delete outside Raphael/src/alexandria;
5. print a concise diff/status summary;
6. fail if source/destination roots are ambiguous or if deleting a stale file would remove untracked user work without a safe policy.

After sync, run the existing parity test, inspect git diff, and commit the embedded copy with the gateway changes. Do not hand-edit embedded src/alexandria to make CI pass.

Step 3 - Prove the built package outside the repository

```
python -m pip install --upgrade build
python -m build
```

Install the wheel in a clean temporary environment outside the source tree and run a small public-API smoke test there. The package must not depend on a developer-specific path or on importing Raphael's repository tree.

Step 4 - Align supported Python versions

pyproject.toml currently declares requires-python = ">=3.10"; CI currently tests 3.10, 3.11, and 3.12. Python 3.10 reached EOL in October 2026. After tests prove compatibility, update requires-python and the CI matrix to maintained versions (proposed initial set: 3.11, 3.12, 3.13). Coordinate this change with Raphael Gateway CI in the same work batch. Do not claim compatibility with an untested interpreter solely because basic import succeeds.

A5 gate

Must: canonical source and embedded package are byte-for-byte in parity; sync is deterministic and safe; clean wheel install/smoke test succeeds. **Should:** maintained Python matrix is aligned across both repos and package metadata.

9. A6 - Recovery drill and Raphael integration

Step 1 - Run provider lifecycle tests

Against a disposable SQLite database, verify this compact sequence:

1. create a valid memory and confirm approved status;

2. repeat the same request ID and payload and confirm idempotency;
3. reuse the request ID with different content and confirm conflict;
4. create a normalized duplicate under a different request ID and confirm only one active row;
5. edit and verify normalization/body hash update;
6. reject editing a missing/deleted row;
7. soft-delete and verify exclusion from approved list and model context;
8. restore within the undo window if no active duplicate exists;
9. verify restore conflicts when an active duplicate exists;
10. verify restore after the undo window is rejected;
11. create and validate a SQLite backup;
12. restore it offline to a different disposable path, reopen via the provider, run integrity/schema checks, and execute a read/write smoke test.

Do not add a JSON import test; JSON restore is deliberately out of scope for Revision 3.4.

Step 2 - Run the backup/restore drill once

For first production release and after any change to backup/restore code:

1. create a fresh SQLite backup from the test/live provider using create_sqlite_backup();
2. verify non-zero size, SHA-256 (optional but useful), integrity and schema;
3. stop the gateway for the restore target (never shut down the real service just to test a disposable path if a separate test DB is available);
4. restore the backup to a new, disposable database path;
5. verify counts/schema and representative synthetic records;
6. create, list, edit and soft-delete a synthetic fact against the restored DB;
7. restart a test gateway with that DB and perform an authenticated memory request;
8. record date, source commit, schema version, backup hash, duration, counts and pass/fail without recording memory bodies or tokens;
9. never overwrite the only live database during this drill.

This is the minimum evidence required for the personal release. Formal RPO/RTO documentation and a full maintenance scheduler are optional unless data-loss tolerance requires them.

Step 3 - Run Raphael integration and privacy tests

From the Raphael repo:

```
Set-Location C:\Dev\Raphael
python scripts\sync_alexandria.py
pytest -q tests\test_alexandria_parity.py
pytest -q
```

Run the exact gateway configuration used for deployment. Verify:

- explicit user save creates an approved fact;
- list/edit/delete/undo behavior is stable;
- deleted facts do not enter prompt context;
- duplicate errors map to stable client-safe API responses;
- Alexandria being unavailable degrades according to documented behavior without losing the chat request;

- complete chat transcripts are never sent to memory routes implicitly;
- logs do not contain test markers, prompt text, memory bodies, tokens, or internal SQLite exceptions.

A6 gate and production definition of done

Must:

- ☐ Alexandria CI and Raphael Gateway CI are green for the release candidate commit.
- ☐ The JSON restore implementation and reachable entry points are removed, or an explicit documented external dependency justifies retaining them.
- ☐ Active-memory uniqueness is enforced by SQLite and cross-provider tests pass.
- ☐ SQLite backup verification and offline restore succeed on a disposable target.
- ☐ Sync output matches the canonical package and clean packaging smoke test passes.
- ☐ Provider lifecycle, privacy, and Raphael integration tests pass.
- ☐ Documentation distinguishes readable JSON export from full SQLite backup and restore.

Should:

- ☐ CI and requires-python declare maintained runtimes consistently across both repositories.
- ☐ Migration upgrade/failure fixtures cover all schema versions that have been shipped.
- ☐ Backup location and a practical retention habit are documented; at least one separate protected copy exists if laptop loss is in scope.

Do not claim that a restore drill, retention automation, or multi-version CI passed unless it was actually run. A README checklist may report these as pending until evidence exists.

10. Commands and source map

Core local commands (Windows PowerShell)

```
# From the Alexandria repository
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e .
python -m pip install -r requirements.txt
pytest -q

# Build package
python -m pip install --upgrade build
python -m build

# From the Raphael repository, after a canonical Alexandria change
Set-Location C:\Dev\Raphael
python scripts\sync_alexandria.py
pytest -q tests\test_alexandria_parity.py
pytest -q
```

Source-of-truth map

Concern	Authoritative location
Provider contract	src/alexandria/provider.py
Domain models and export types	src/alexandria/models.py
Content/secret/transcript validation	src/alexandria/validation.py
Transactions, idempotency, duplicate handling	src/alexandria/sqlite_provider.py
Ordered schema changes	src/alexandria/migrations.py
SQLite backup/restore and optional JSON export	src/alexandria/backup.py
Provider/lifecycle tests	tests/test_provider.py
Validation tests	tests/test_validation.py
Migration failure/upgrade tests	tests/test_migrations.py
Backup/restore tests	tests/test_backup.py
Independent provider/connection tests	tests/test_concurrency.py
Alexandria CI	.github/workflows/alexandria.yml
Raphael synchronization tool	Raphael/scripts/sync_alexandria.py
Gateway parity test	Raphael/tests/test_alexandria_parity.py

11. Official technical references

1. SQLite Online Backup API: https://www.sqlite.org/backup.html
2. SQLite write-ahead logging: https://www.sqlite.org/wal.html
3. SQLite partial indexes: https://www.sqlite.org/partialindex.html
4. Python sqlite3 transaction control: https://docs.python.org/3/library/sqlite3.html
5. Python release lifecycle/current supported versions: https://www.python.org/downloads/
6. Python packaging and build: https://packaging.python.org/en/latest/tutorials/packaging-projects/
7. Raphael repository: https://github.com/Agri99/Raphael

Canonical roadmap reminder: complete A0 first, then A1-A6 in order. Prefer removing a risky unused feature to adding a second layer of validation and operational complexity. Markdown is the source of truth; DOCX/PDF copies are generated from this document.